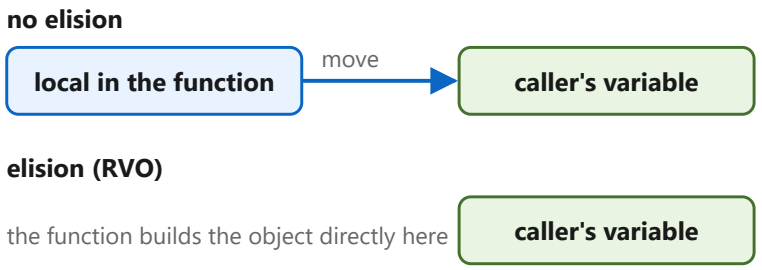
Returning Objects and Copy Elision

Why returning a big object by value is cheap: RVO, NRVO, guaranteed elision in C++17, and the few cases where `std::move` on a return helps. Assumes Writing Move Operations.

Returning a local: the compiler does the work

Returning an object by value sounds expensive. In practice the compiler removes the cost: it constructs the returned object **directly in the storage of the caller**, so no copy or move happens at all. This is **copy elision**, also called RVO (return value optimization) and, for a named local, NRVO.



Elision builds the result where the caller wants it: zero copies, zero moves.

```
// returns a temporary: elision is guaranteed
Buffer make()          { return Buffer(1000); }
// returns a named local: NRVO, allowed and done in practice
Buffer make2()         { Buffer b(1000); return b; }
// built in place: 0 copies, 0 moves (compiled)
Buffer big = make();
```

SO WRITE IT PLAINLY Return locals and temporaries by value, with no `std::move`. The compiler either elides it or, if it cannot, moves it automatically.

Guaranteed elision in C++17

Since C++17 the elision of a **temporary** is mandatory, not an optimization. The object is never copied or moved, so the type does not need a copy or move constructor at all.

`return Pinned();`
C++17: no copy or move exists

`Pinned(const Pinned&) = delete;`
Pinned(Pinned&&) = delete; still compiles

Before C++17 this needed an accessible move or copy constructor, even if the compiler skipped it.

Guaranteed elision is not an optimization: it is the language rule.

```
struct Pinned {
    Pinned() = default;
    Pinned(const Pinned&) = delete;
    Pinned(Pinned&&)      = delete;
};
// compiles in C++17 (compiled): guaranteed elision
Pinned make() { return Pinned(); }
Pinned p = make();
```

KIND OF RETURN	ELISION	IF THE COMPILER DOES NOT ELIDE
`return Type( ... );` (a prvalue)	**guaranteed** since C++17	not possible: nothing to copy or move
`return localName;` (NRVO)	allowed, done by all major compilers in simple cases	the local is **moved** automatically, never copied

When `std::move` on a return is right and wrong

Compiled with a counting type (copies and moves performed), these are all the common forms:

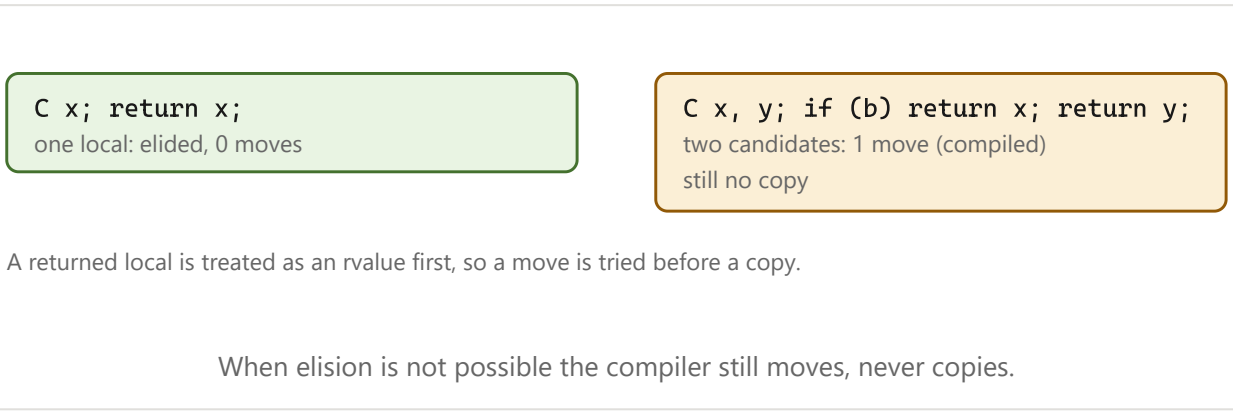
WHAT YOU WRITE	COPIES	MOVES	WHY
`return Counter();`	0	0	guaranteed elision: built in the caller's storage
`Counter c; return c;`	0	0	NRVO: allowed, not required, done in practice
`return std::move(c);`	0	**1**	blocks NRVO: an extra move, so never write it
return a by-value **parameter**	0	1	no elision for parameters, but it is moved, not copied
`return member;`	**1**	0	a member is never moved implicitly
`return std::move(member);`	0	1	here `std::move` is right: the object keeps living, you take its member

```
// fine: elided or moved automatically
Buffer make()           { Buffer b(1000); return b; }
// member of an object that outlives the call: move it out
Buffer take(Source& s) { return std::move(s.buffer); }
```

> RULE Return locals and temporaries plainly. Use `std::move` on a **member**, or on something that is not the local you are returning by name. `-Wpessimizing-move` (in `-Wall`) warns about the wrong one.

When elision does not happen

NRVO is allowed, not required, and it needs the compiler to know *which* object is the result. When it cannot decide, it falls back to a **move**: a returned local is treated as an rvalue first.




```
// NRVO: 0 copies, 0 moves (compiled)
C plain()      { C x; return x; }
// two candidates: 0 copies, 1 move (compiled)
C pick(bool b) { C x, y; if (b) return x; return y; }
```

RETURN FORM	RESULT
a single named local on every path	NRVO, no move
different locals on different paths	usually one move
a by-value parameter	one move, never elided
a member or global	one copy, unless you `std::move` a member that you own
`return cond ? a : b;` with named objects	a copy: the conditional expression is an lvalue

DO NOT FIGHT IT Write the natural code. If a profile shows a move you want to avoid, build the result in the caller (pass an output parameter) rather than adding `std::move`.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
When std::move on a return is right and wrong	Effective Modern C++	Item 25, Use std::move on rvalue references, std::forward on universal references	186 to 195
	C++ Move Semantics	4.1 to 4.2 Avoid objects with names; avoid unnecessary std::move	77 to 79